Tesi di Laurea in
Image Processing and Machine Vision

A DEEP LEARNING SOLUTION FOR INDUSTRIAL OCR APPLICATIONS

CANDIDATO

Lorenzo Lamberti

RELATORE

Chiar.mo Prof. Luigi Di Stefano

CORRELATORI

PhD Luis Gongalves

Dott. Filippo Mambelli

DATALOGIC

OBIETTIVO TESI

Riconoscimento di caratteri

Leggere e ricostruire stringhe di caratteri alfanumerici da immagini

DEEP LEARNING

Sottocategoria Machine Learning



OCR Industriale

Riconoscimento ottico dei caratteri

RETI CONVOLUZIONALI

Algoritmo You Only Look Once (YOLO)
General purpose object detector

OCR

Optical Character Recognition

AMBIENTE NON CONTROLLATO

Scenari variabili, imprevedibili
Condizioni di luce estreme
Variazione grandezza e font caratteri

PERFORMANCES

Applicazioni per consumer
Non deve essere infallibile



AMBIENTE CONTROLLATO

Setup camera fisso
Condizioni di luce ideali
Grandezza e font caratteri costante

VINCOLI DI ACCURATEZZA

Estremamente alti (> 99%)

VINCOLI DI VELOCITÀ

Esecuzione in tempo reale

DISPONIBILITÀ DEI DATI SCARSA

L'applicazione è specifica e i dati vanno raccolti ogni volta direttamente sul campo

Neural Networks

01

Architettura a strati

Singole unità computazionali ripetute: neuroni.

Struttura a strati: un input e un output layer, molteplici layer nascosti.

02

Completamente connessa

Ogni neurone è collegato a tutti i neuroni dello strato precedente

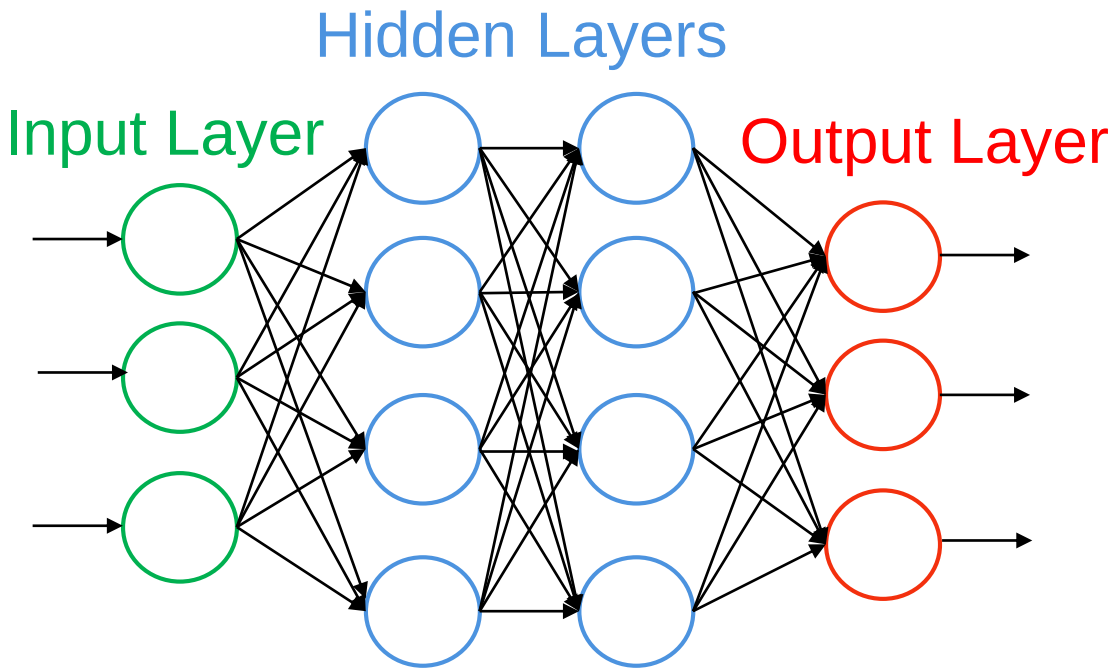
03

Modello non-lineare

Funzione di risposta composta.

È una black-box.

Modella relazioni non lineari



Convolutional Neural Networks

Pensate per immagini

Dati con struttura matriciale.

Differenze con reti Fully-Connected

Struttura 3-D

Neuroni creano strutture a tre dimensioni.
Rappresentabili come filtri 2D sovrapposti.



Connessione Locale

Ciascun neurone appartenente ad un livello non è più collegato a tutti i neuroni del livello precedente.

Idee di base

Condivisione dei Parametri

Parametri sono utilizzati più volte in uno stesso layer

Filtri Convoluzionali

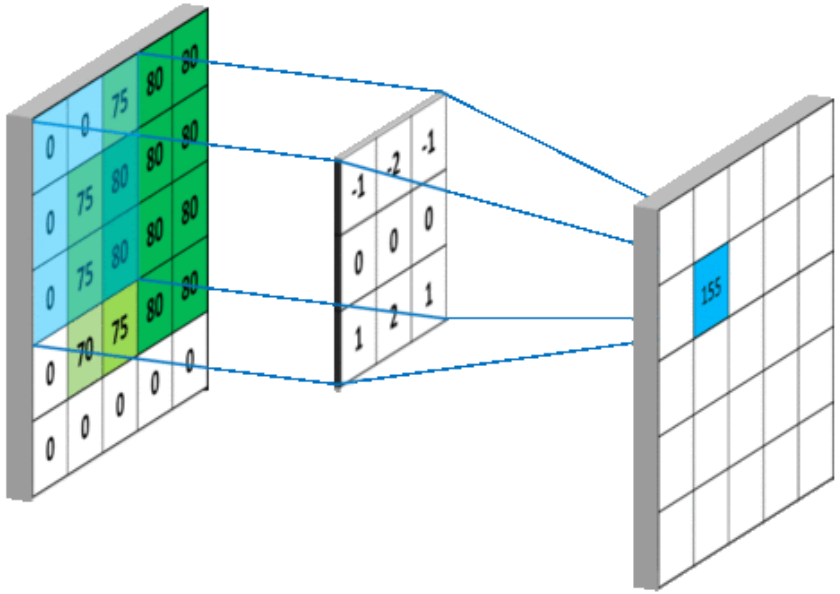
Output singolo valore.
Convoluzione tra input e filtro 3D.
Ad ogni step fatta una somma pesata.

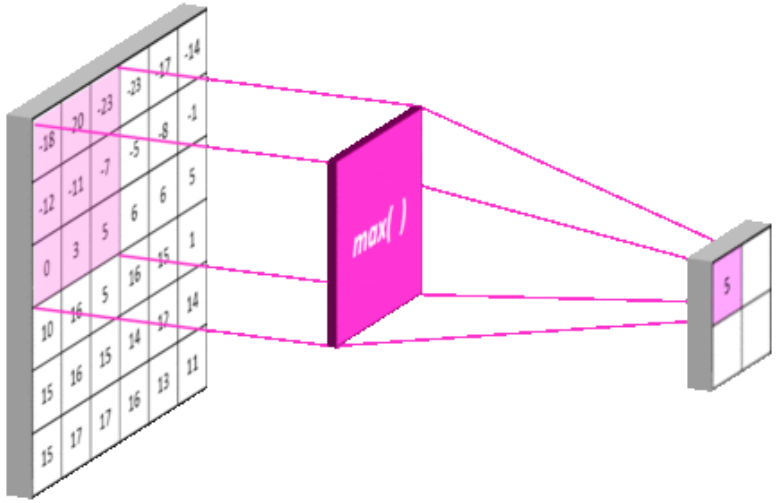
Tipi di Filtri

Filtri Max-Pooling

Diminuisce la dimensione della rappresentazione dei dati. Ogni volta facendo down-scaling.
Caratteristiche basso-livello vs generali alto-livello

Convolutional Neural Networks





Tipi di Filtri

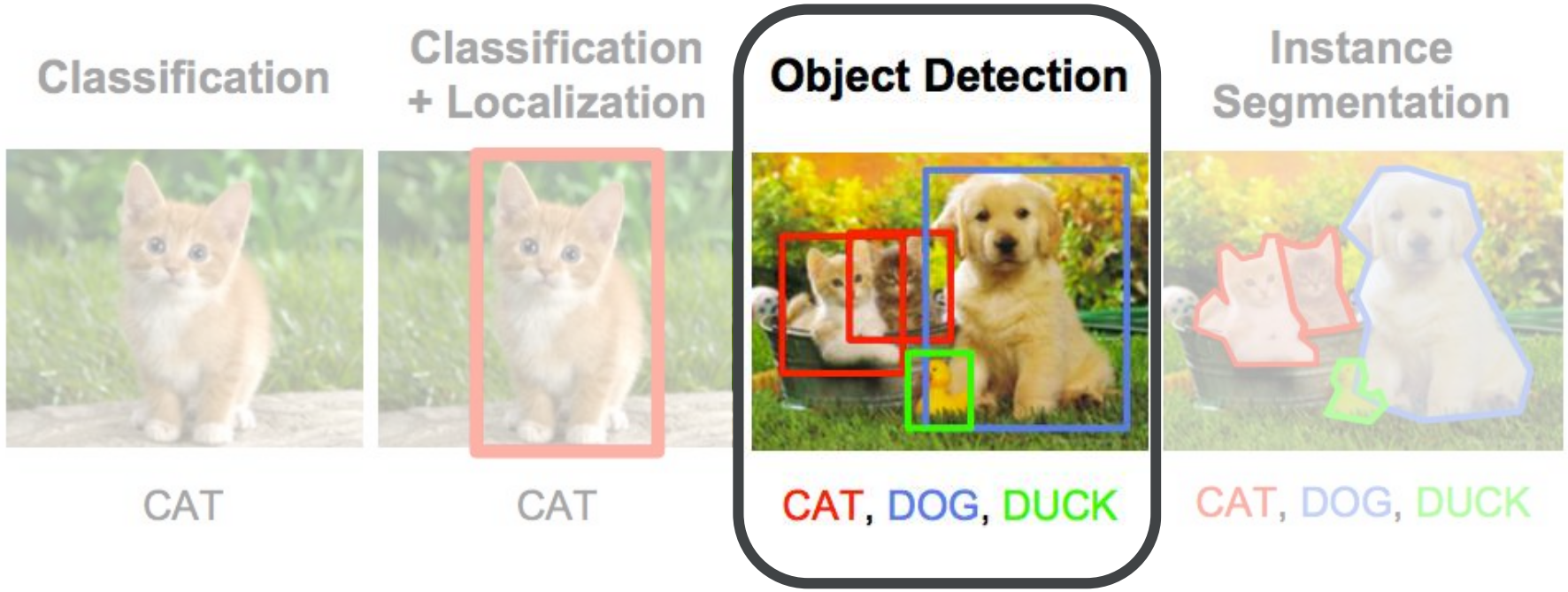
Filtri Convoluzionali

Output singolo valore.
Convoluzione tra input e filtro 3D.
Ad ogni step fatta una somma pesata.

Filtri Max-Pooling

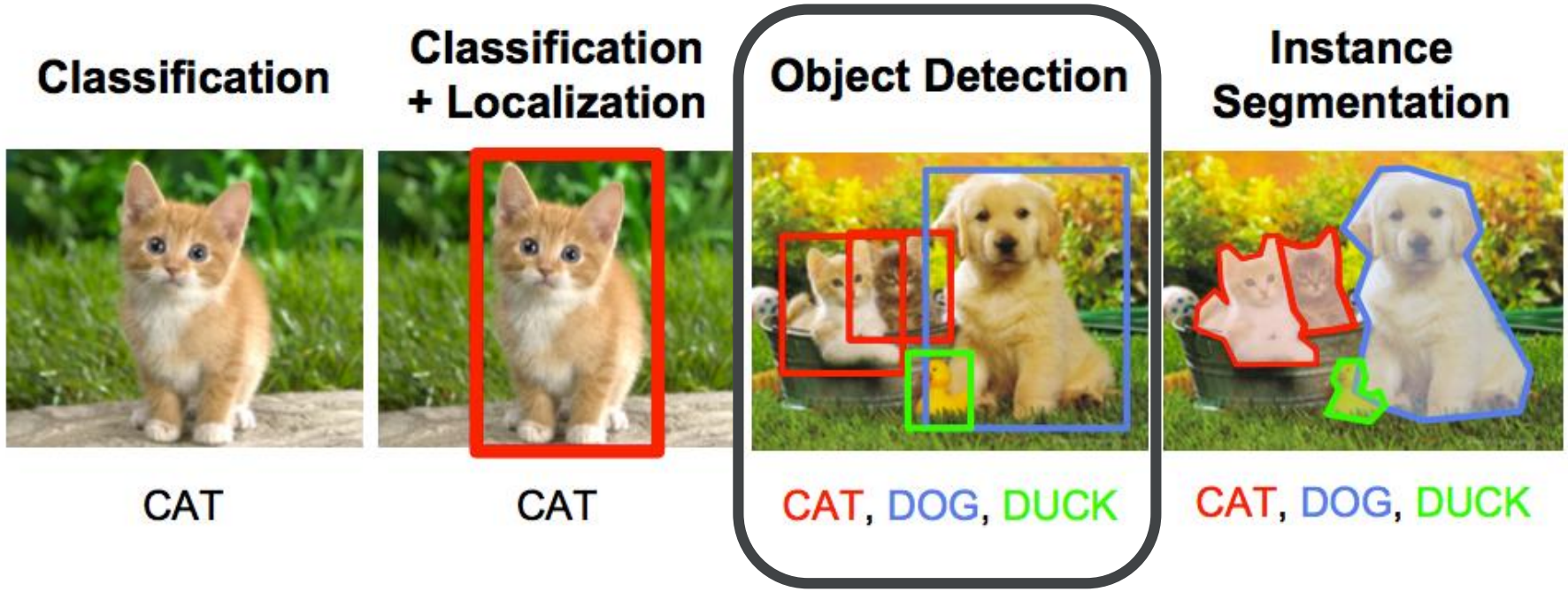
Diminuisce la dimensione della rappresentazione dei dati. Ogni volta facendo down-scaling.
Caratteristiche basso-livello vs generali alto-livello

Object Detection



LOCALIZZAZIONE	CLASSIFICAZIONE
Regressione delle coordinate e della dimensione bounding box	Etichettare ciascuna box

Object Detection



LOCALIZZAZIONE	CLASSIFICAZIONE
Regressione delle coordinate e della dimensione bounding box	Etichettare ciascuna box

YOLO: You Only Look Once

01

Rete Convoluzionale

Usa una struttura convoluzionale invece che a sliding-window

02

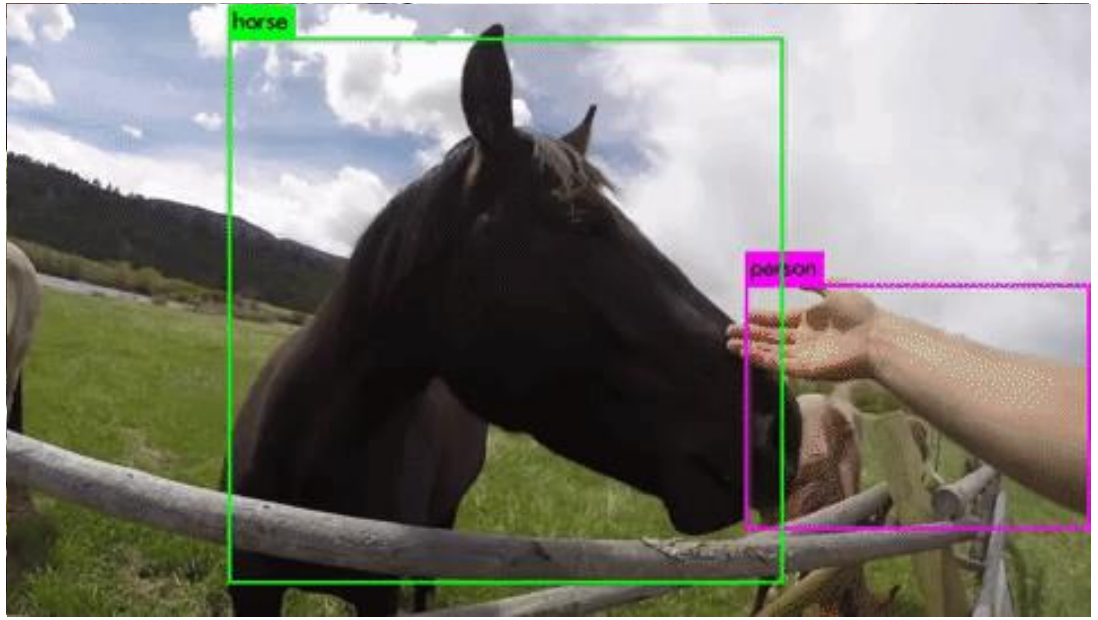
Single-Step Object Detector

Esegue regressione della posizione e classificazione in un solo colpo

03

Real-Time Performances

Estremamente veloce, adatto per elaborazione real-time



04

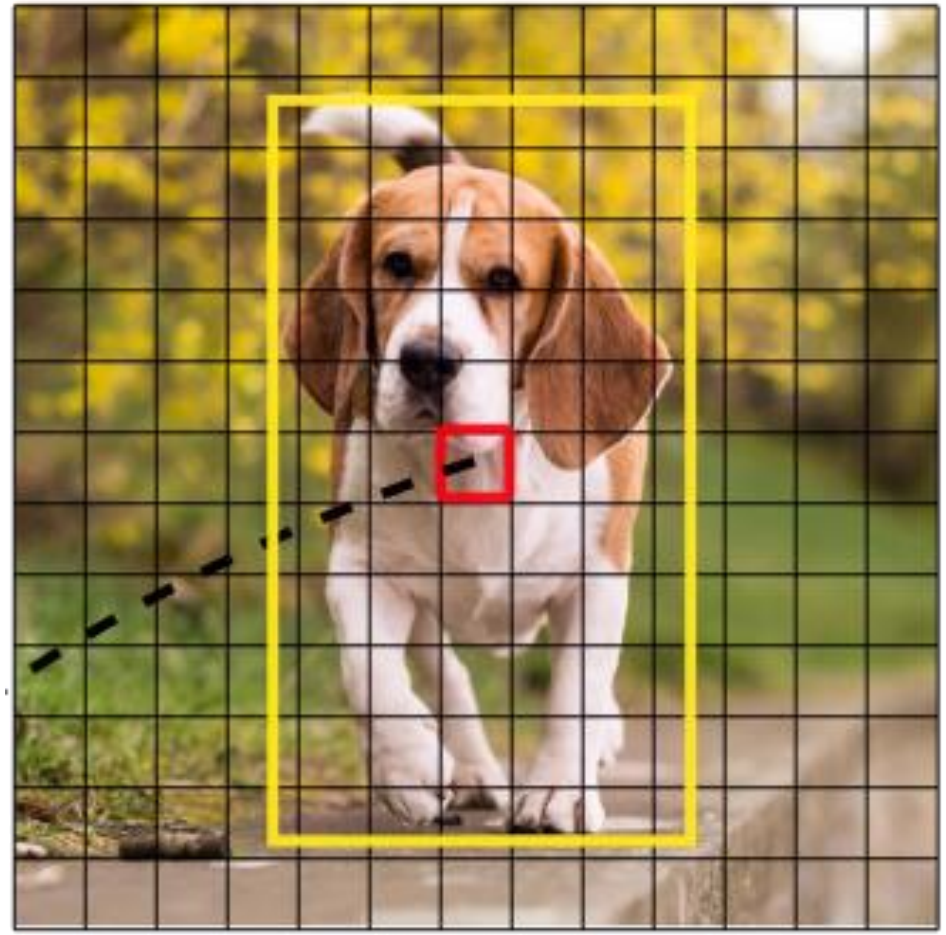
Problema: difficoltà a riconoscere oggetti piccoli e raggruppati

È proprio il caso dell'OCR.

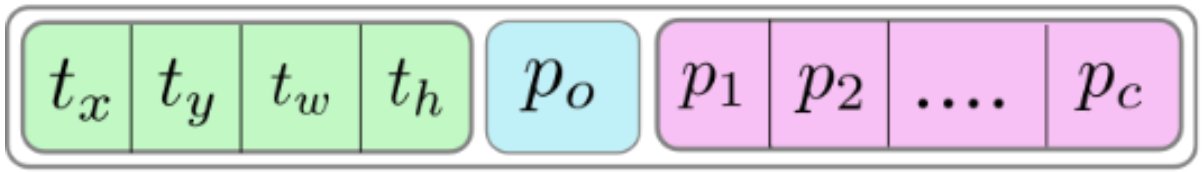
Sfida: sarà sufficientemente accurato in questo campo?

Struttura Output

Divisione immagine in una **griglia**
Dimensione dipende dall'architettura della rete



B x



01 **Coordinate e dimensioni box**
2 coordinate spaziali
2 dimensioni della bounding box (altezza e larghezza)

02 **Objectness score**
Confidenza dell'algoritmo di aver predetto davvero un oggetto e non sfondo

03 **Classification score**
Punteggio per la classificazione tra le categorie

Oggetti piccoli e raggruppati

Il numero di box predette per ogni cella è un **parametro settabile**

In presenza di più caratteri in un'unica cella non c'è modo di garantire che non ci siano N predizioni tutte sullo stesso carattere

Motivo per cui YOLO ha fatica a riconoscere più oggetti quando sono più piccoli della cella stessa

Griglia più densa
Almeno una cella per carattere
Una predizione per cella

Esempio griglia **non abbastanza densa**



Implementazioni di YOLO

V1

- 24 layer convoluzionali
- **2 layer fully connected**
- **Può predire solo un tipo di classe per cella**
- **45 FPS** sul dataset PASCAL

V2

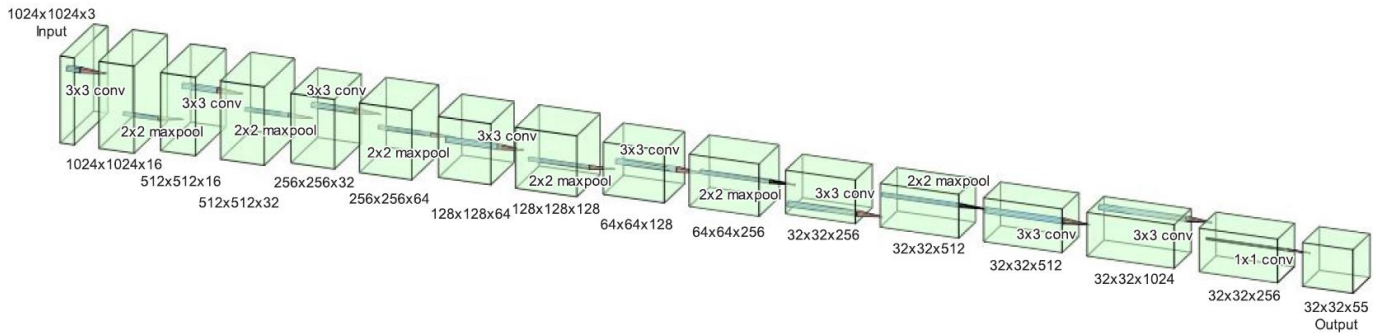
- 19 layer convoluzionali
- **Completamente convoluzionale**
- **Ogni box predetta classificata individualmente**
- Regressione delle coordinate a partire da modelli
- **91 FPS** sul dataset PASCAL

V3

- 106 layer convoluzionali
- 3 predizioni su scale crescenti per predirre su scale diverse
- **20 FPS** sul dataset COCO

Architettura custom basata su YOLOv2:

- **Ridotta**: 7 layer convoluzionali
- I caratteri hanno forme semplici: rete piccola
- **125 FPS** con immagini 550x550



Attività di riconoscimento

RICONOSCIMENTO CARATTERI

Individuare e classificare correttamente ogni singolo carattere



INDIVIDUAZIONE STRINGA

Localizzare una particolare stringa di interesse per processarla successivamente



Metriche: categorie di predizione

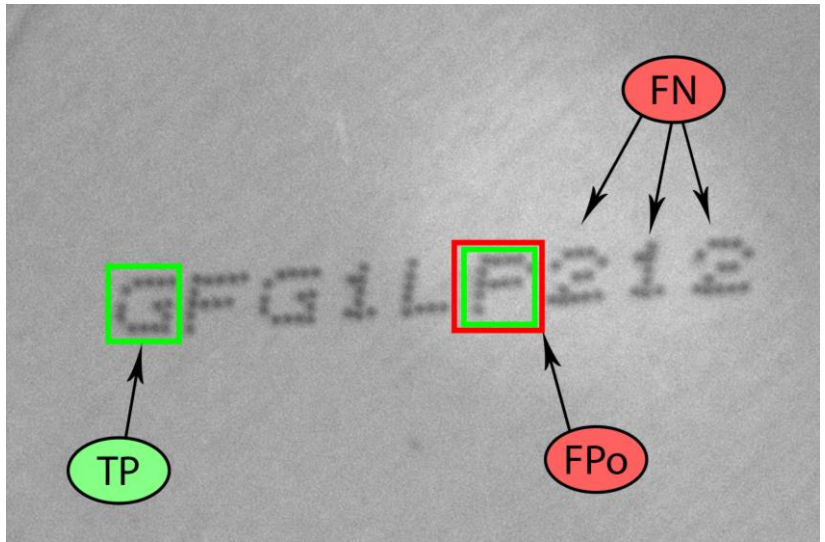
Riconoscimento caratteri

FN

False Negative
Mancata rilevazione carattere

FPo

False Positive Overlapping
Carattere classificato incorrettamente si sovrappone ad una predizione corretta



Individuazione stringa

LineMiss

Line Miss
Mancata rilevazione stringa

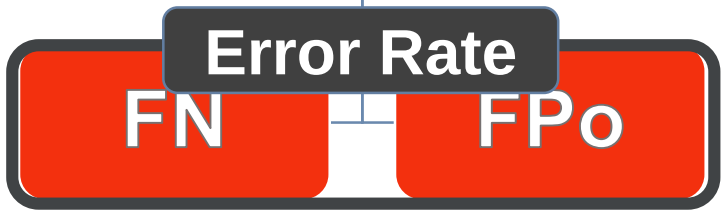
CharMiss

Char Miss
La box predetta non include tutta la stringa al suo interno



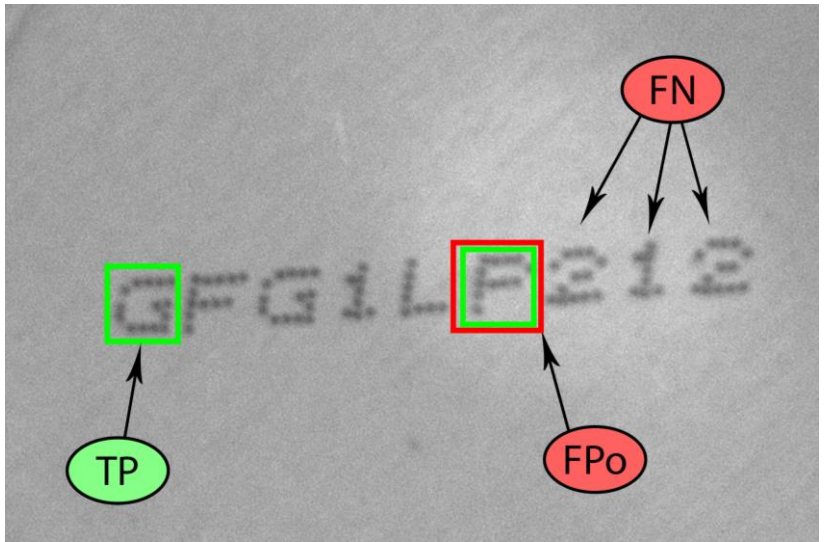
Metriche: categorie di predizione

Riconoscimento caratteri



False Negative
Mancata rilevazione carattere

False Positive Overlapping
Carattere classificato incorrettamente si sovrappone ad una predizione corretta



Individuazione stringa



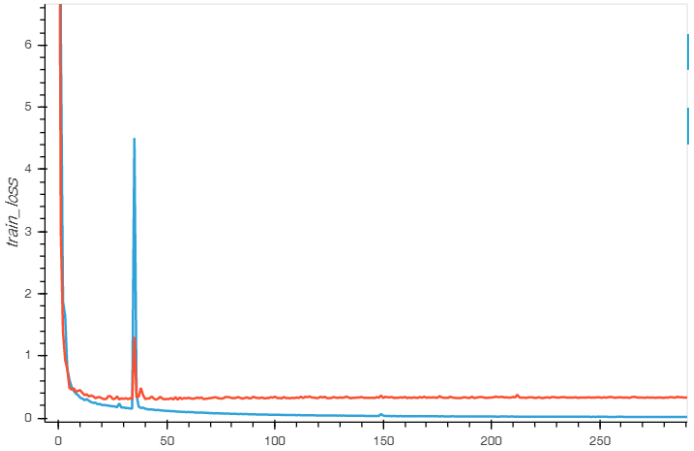
Line Miss
Mancata rilevazione stringa

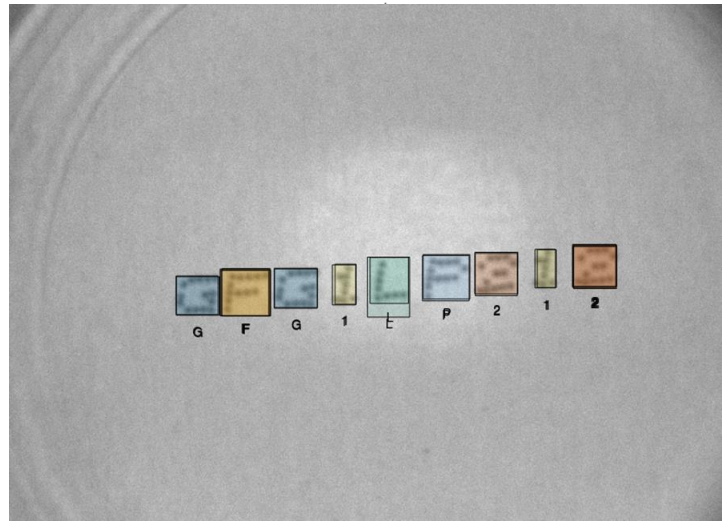
Char Miss
La box predetta non include tutta la stringa al suo interno



Dataset 1:

Semplice, tutte le immagini hanno la stessa sequenza





Loss Function

$$\lambda_{coord} = 2.0$$

$$\lambda_{class} = 1.0$$

$$\lambda_{obj} = 1.0$$

$$\lambda_{noobj} = 0.2$$

Iperparametri

Dimensione immagini: 1280x1024

Dimensione Batch: 8

Ottimizzazione: AdaDelta

Predizioni per cella: 1

Dataset

789 training

200 validation

8876 testing

Confronto tra nuova implementazione Python e originale in C:

Python					
	FPno rate	FPo rate	FPso rate	FN rate	Image Level Error Rate
Cell Size = 32 Predizioni box = 1	0.54%	0.10%	0.00%	0.15%	**0.25%**

C				
FPno rate	FPo rate	FPso rate	FN rate	Image Level Error Rate
0.16%	0.30%	0.00%	0.06%	**0.09%**

Velocità di Elaborazione: 70 FPS (Real-time)

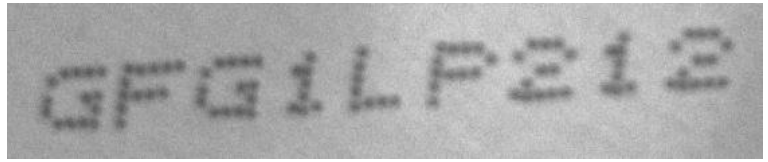
Memorizzazione

Dataset 1: **stessa sequenza** di caratteri ripetuta ogni immagine.

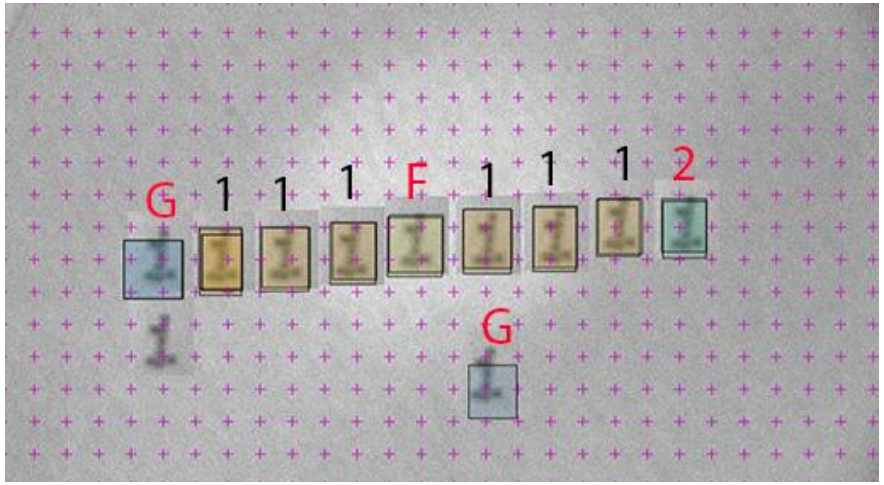
Sequenza imparata:

G F G 1 L P 2 1 2

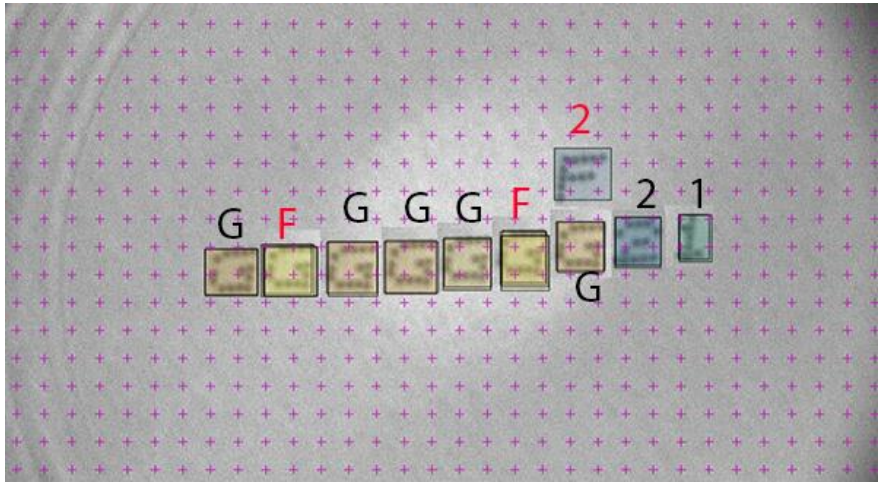
Sequenza originale



Create **immagini fittizie** con sequenze diverse per testare effetti di memorizzazione:



Errori memorizzazione posizione assoluta:
primo e ultimo carattere sempre riconosciuti come G e 2



Errori memorizzazione posizione relativa:
Tra due G impara che c'è sempre una F

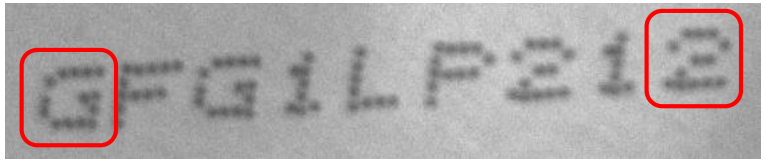
Memorizzazione

Dataset 1: **stessa sequenza** di caratteri ripetuta ogni immagine.

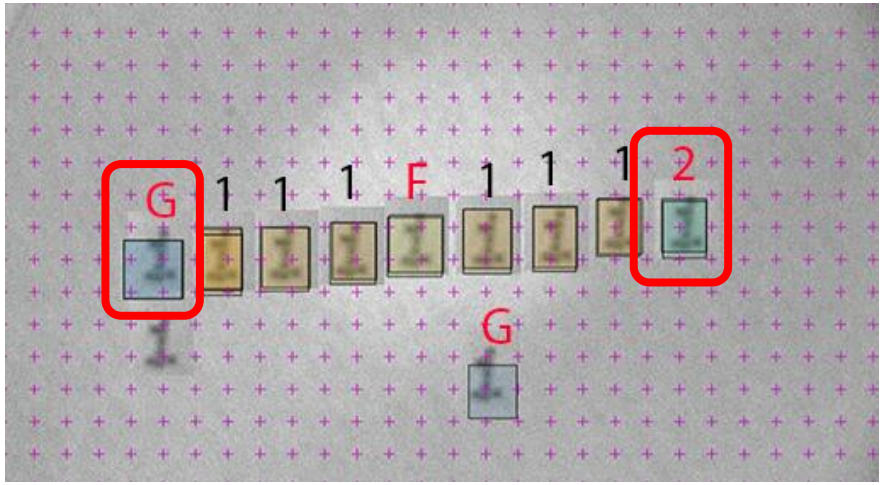
Sequenza imparata:

G F G 1 L P 2 1 2

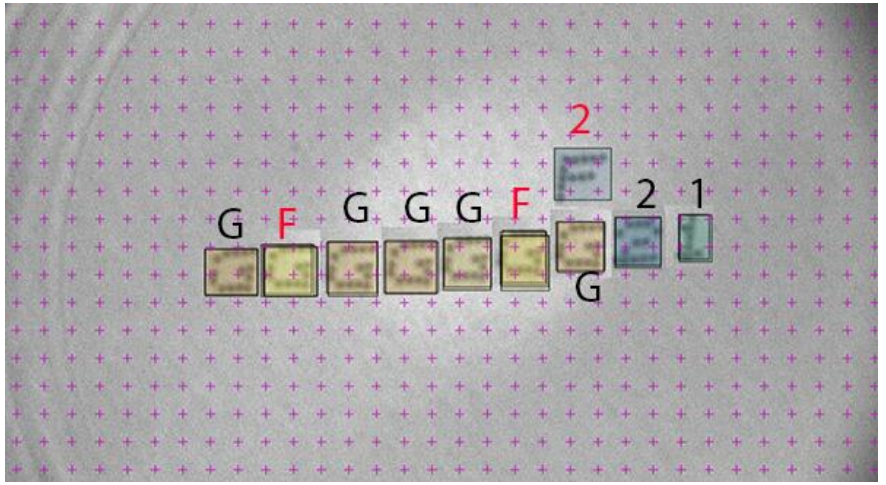
Sequenza originale



Create **immagini fittizie** con sequenze diverse per testare effetti di memorizzazione:



Errori memorizzazione posizione assoluta:
primo e ultimo carattere sempre riconosciuti come G e 2



Errori memorizzazione posizione relativa:
Tra due G impara che c'è sempre una F

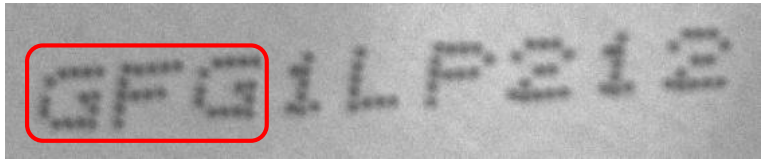
Memorizzazione

Dataset 1: **stessa sequenza** di caratteri ripetuta ogni immagine.

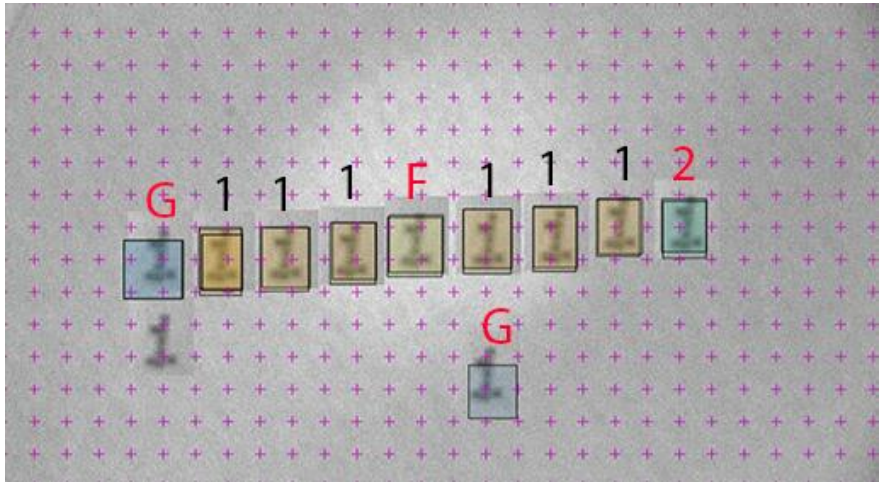
Sequenza imparata:

G F G 1 L P 2 1 2

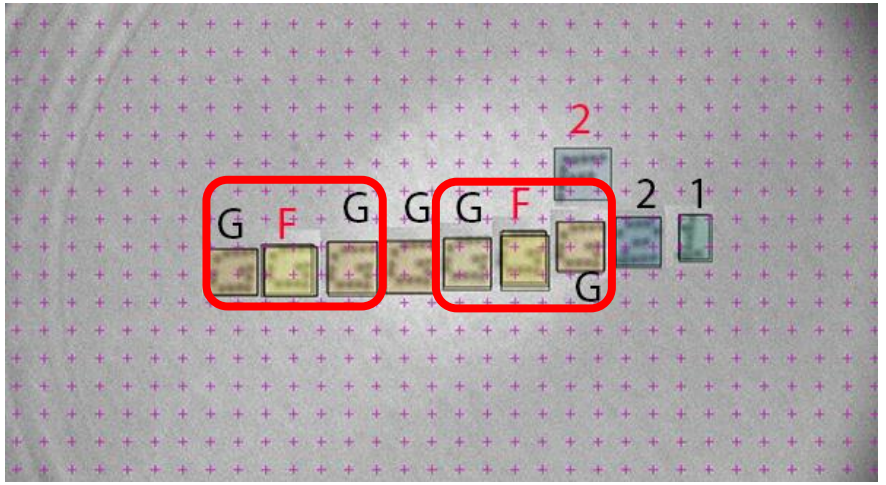
Sequenza originale



Create **immagini fittizie** con sequenze diverse per testare effetti di memorizzazione:



Errori memorizzazione posizione assoluta:
primo e ultimo carattere sempre riconosciuti come G e 2

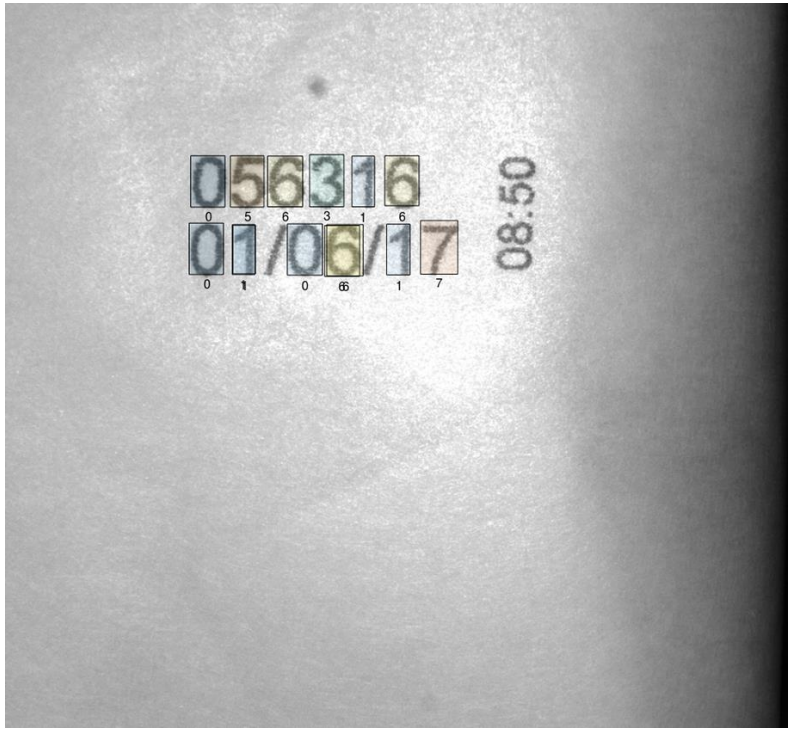


Errori memorizzazione posizione relativa:
Tra due G impara che c'è sempre una F

Dataset 2

Le immagini hanno ora tutte **sequenze diverse**

Testiamo la capacità di YOLO **generalizzare** su nuove sequenza



Prestazioni molto buone, le migliori tra tutti i dataset:

Dataset
1114 training
446 validation
5867 testing

	FPno	FPo	FPso	FN	Image Level Erros
Cell Size = 32 Predizioni box = 1	8	7	1	1	**2/5865**
	0.01%	0.01%	0.00%	0.09%	**0.03%**

Velocità di Elaborazione: 67 FPS

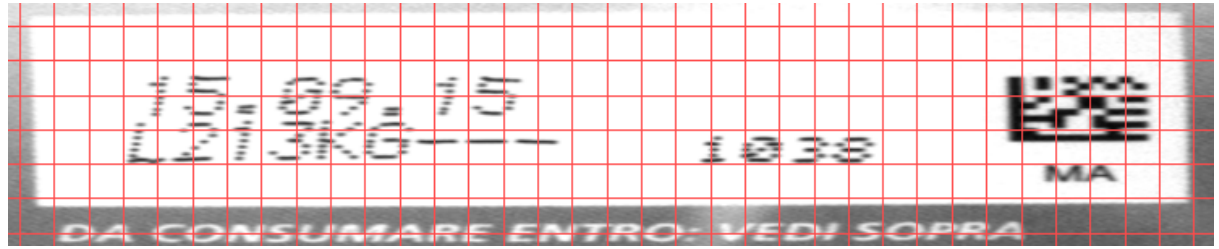
Dataset 3

Le immagini hanno **sequenze diverse**

Lo sfondo varia considerevolmente

I caratteri sono molto fitti

Stretching immagine



Vengono allungate le immagini da 600 x 500 a 1800 x 500:

Più spazio tra i caratteri

Le celle della griglia ha dimensione costante rispetto la dim immagine

La griglia diventa più fitta in rapporto ad un immagine più grande

Esempio predizione



Dataset
2336 training
779 validation
35820 testing

FPno	FPo	FPso	FN	Image Level Erros
58	37	29	74	**90/35816**
0.16%	0.09%	0.01%	0.28%	**0.25%**

Velocità di Elaborazione: 37 FPS

Dataset 4

Immagini ampiamente distorte, sfocate e dense di altri caratteri.

I caratteri sono **molto piccoli** 10x15 pixels.

17 caratteri da riconoscere senza sbagliarne uno.

Il dataset è estremamente difficile: error rate attesa circa 30%

Architettura della rete modificata: tolto layer di max-pooling.

Modifica la **struttura della griglia generata**: celle da 32 a 16 pixel: griglia molto fitta.

Immagini allungate come prima

Stretching immagine



Esempio predizione



Dataset

2777 training

490 validation

4896 testing

FPno	FPo	FPso	FN	Image Level Erros
184	120	223	228	**291/4080**
4.51%	1.24%	0.16%	0.31%	**7.1%**

Velocità di Elaborazione: 63 FPS

Individuazione Stringa

Vogliamo individuare il **tracking number.**

Dataset 4 denso di altre stringhe e informazioni

Margine sulla ground truth durante il training:

YOLO impara distinzione con sfondo

Predizioni ingrandite, probabilità di successo maggiore:

Tutti i caratteri devono stare dentro la box predetta

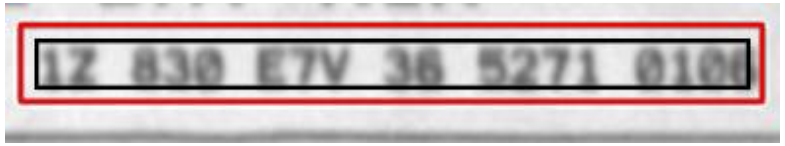
Margine non ampio (20% e 40%) e prestazioni ottime:

Dataset
7791 training
490 validation
6059 testing

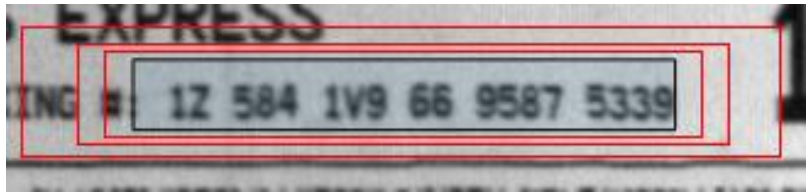
bbox increase: (20%,40%)				
FPno	**CharMiss**	**Strong CharMiss**	**Line Miss**	**Image Errors**
13	61	1	2	**3/6059**
0.23%	1.06%	0.02%	1.06%	**0.05%**

Velocità di Elaborazione: 120 FPS

Margine ground-truth



Margine incrementale predizioni





Test Label Intera

Training sulle etichette croppate: più piccole, training più veloce

Testing sulle label intere per generalizzare



Conclusioni



Performances

Accuratezza: 99.75% - 99.97%

Velocità: 37 - 120 FPS (real time)

Effetti di Memorizzazione

Se non c'è abbastanza variabilità nel dataset training, YOLO impara le sequenze dei caratteri

Due diverse applicazioni

Efficace per individuare:

- Caratteri singoli
- Regioni di interesse intere

Prossimi passi

Algoritmo portato avanti dall'azienda

- Training su dati sintetici
- Test su immagini vere

Grazie per l’attenzione

